

Placeholder

Luka Lasic

Abstract

The performance gap between managed-runtime languages and native code remains a nuanced but practically significant concern for systems where both are viable options. Rather than revisiting the classic space-time tradeoff between garbage collection and explicit memory management, we use the porting of Java benchmarks to native equivalents as a methodology for directly quantifying this gap and understanding its origins. Using modern LLMs to translate eight DaCapo 23.11 benchmarks into workload-equivalent C and C++ implementations (potentially Rust?), we measure execution time, memory footprint, and microarchitectural behavior across a range of Java configurations and native baselines. (Mention prior work or just ignore prev paper?) This study finds that the Java/native gap is substantially larger than what was found before with a $2.53\times$ to $26.35\times$ in overall execution time. This study will aim to investigate what drives the gaps found whether they are artifacts of GC overhead specifically, or whether they reflect deeper differences in compilation strategy, memory layout, runtime abstraction costs, workload characteristics, or LLM simplification.

1 Introduction(Placeholder from Prev Paper)

Garbage collection (GC) is a defining feature of managed runtime systems such as Java. By automating memory reclamation, GC relieves programmers of the burden of precisely managing their own memory, allowing them to avoid memory-management bugs. However, these benefits come with known costs: GC consumes execution time, often requires additional memory headroom, and can alter cache behavior, locality, and pause-time characteristics. The main cost of GC is the overhead of garbage collection itself. Explicit memory management (EMM), by contrast, offers direct control over allocation and reclamation, but shifts complexity and correctness burdens back to the programmer.

A quantitative study aimed at understanding the GC vs. EMM tradeoff is Hertz and Berger’s 2005 OOPSLA paper [1]. To avoid comparing different source languages or differently structured programs, they introduced an *oracular* memory manager that executed unmodified Java programs as if they used explicit memory management. Their results provided a key insight: GC was competitive with explicit memory management only when given substantially more memory, matching or slightly exceeding EMM when given roughly 5× the required memory, slowing by about 17% at 3×, and by about 70% at smaller heap sizes on average [1]. They further showed that explicit memory management generally achieved better memory utilization and page-level locality [1]. Their results have had a lasting influence on how researchers and practitioners think about the performance costs of automatic memory management, and the debate continues to this day.

Twenty years later, however, it is no longer clear that the same conclusions hold for modern systems. The collector landscape has changed substantially. Contemporary JVMs now include mature region-based and highly concurrent collectors such as G1 and ZGC [8, 18, 19]. These collectors shift work concurrently and generationally to reduce pause times, but doing so introduces new trade-offs in barrier costs, metadata, and CPU overhead [8]. At the same time, modern work suggests that GC costs are more multidimensional. Cai et al. [3] show that, even in OpenJDK 17, production collectors can still incur substantial overhead relative to a distilled no-GC baseline, with reported lower bounds of 7–82% more wall-clock time and 6–92% more CPU cycles at a modest

heap size. Sareen and Blackburn [9] likewise show that the modern picture is more nuanced than a simple “GC is slow.” In modern allocators, GC can have little impact on mutator time in some settings, while space overhead varies substantially by collector design, from roughly 65–80% for simple copying collectors to 11–17% for Immix, relative to a baseline approximating manual memory management. In addition, more up-to-date benchmark suites such as DaCapo provide a new baseline against which to compare earlier results.

There have also been improvements on the EMM side. Hertz and Berger were compared against a general-purpose allocator like LEA, which was appropriate at the time, but current systems frequently rely on highly engineered general-purpose allocators such as jemalloc and mimalloc. Each is designed to improve scalability, reduce contention, and enhance page locality [11, 13, 14, 15].

This paper revisits the GC-versus-EMM question using a different methodology. Rather than attempting to retrofit explicit reclamation into a contemporary JVM, we compare modern Java executions against workload-equivalent native re-implementations of selected DaCapo benchmarks in C. We focus on the DaCapo 23.11 workloads as a more modern benchmark corpus. This choice is intended to preserve algorithmic and workload comparability while avoiding many of the validity threats that a JVM-internal oracle replay may introduce, as previous work argues against [3]. Our goal is not to claim a pure isolation of GC cost in the strong sense achieved by the original oracle experiment. Instead, we ask a narrower but still important modern question: under contemporary collectors, allocators, workloads, and hardware, how closely do Java executions approach comparable native baselines?

To answer that question, the paper is organized around seven research questions:

RQ1. How do modern Java collectors compare with native baselines in overall runtime?

RQ2. Does giving Java more heap space close the performance gap to native execution?

RQ3. How do Java and native executions differ in resident memory footprint and dynamic work?

RQ4. What microarchitectural differences accompany the observed runtime gap?

RQ5. How much does collector choice matter once Java is tuned fairly?

RQ6. What do the C++ probe implementations reveal about the role of implementation structure?

RQ7. How sensitive is the observed Java/native gap to optimization of the native baseline?

2 Related Work(Place holder but will probaly keep most of this refernced work, maybe one paper about LLM porting since it is relevant)

The central study that informs our research is Hertz and Berger’s 2005 study on garbage collection versus explicit memory management [1]. Their key contribution was to introduce an oracular memory manager that enabled Java programs to execute with `malloc/free` semantics under oracle guidance. Their results showed a clear space–time tradeoff and served as the main baseline for our work. However, to understand what has changed, we must examine more recent work.

A study by Cai et al. [3] suggests that the collector side of the original study has changed substantially. Cai et al. [3] argue that the absolute cost of GC remains insufficiently understood and that the Hertz & Berger methodology does not transfer cleanly to modern production runtimes. Using Epsilon as a no-op baseline, they develop a methodology that places a lower bound on GC overhead in OpenJDK 17 and show that modern production collectors still incur substantial costs. Across a diverse suite of modern benchmarks and at a modest heap size, they estimate that production GCs incur at least 7–82% more wall-clock time and 6–92% more CPU cycles than the distilled baseline. Averaged over 16 benchmarks at 3.0× heap, their lower-bound overheads are 1.13, 1.06, 1.09, 1.43, and 1.54 in time for Serial, Parallel, G1, Shenandoah, and ZGC, respectively, and 1.06, 1.10, 1.14, 1.47, and 1.68 in CPU cycles. Research on modern collectors provides some

support for our own reexamination of such collectors to determine whether giving more heap space improves performance relative to an explicit allocator.

Zhao and Blackburn [8] deconstruct a modern collector, G1, into its major mechanisms and quantify the costs and benefits of each. Their analysis shows that modern GCs hide multiple underlying tradeoffs in concurrency, barriers, remembered sets, and evacuation strategy. In particular, they find that concurrent marking and generational collection reduce 95th-percentile GC pause times by 64% and 93%, showing how much modern pause-time improvements depend on concurrency and generation-aware collection. These results are relevant to our study because they show that modern GC performance reflects a bundle of design choices in concurrency, barriers, remembered sets, and evacuation strategies.

Sareen and Blackburn [9] broaden the picture further by re-examining the costs and benefits of automatic memory management using modern allocator methodologies. Their results show that space overhead is highly collector-dependent and that some commonly repeated interpretations of Hertz and Berger’s space-overhead result overstate what that paper directly established. Their analysis matters directly to our study because it suggests that a modern revisit requires us to examine collector overhead, which varies substantially by design. Thus, a fair study must use modern collectors and current explicit allocators.

Carpen-Amarie et al. [6] add an important cache-level perspective, showing that concurrent GC can cause severe cache disruption in cache-sensitive workloads but only modest effects on most modern server benchmarks. Their discovery is important to us because it means that some benchmarks may expose substantial cache and concurrency penalties, while others may largely hide them. Therefore, our benchmark ports must be workload-equivalent and must examine results beyond runtime alone.

Taken together, prior work leaves open whether modern Java and modern workloads still exhibit the same space–time tradeoff that has been observed before under a new cross-language comparison using native ports.

3 Methodology(Will remain largely unchanged for now)

3.1 Port Construction

The native benchmark suite was built through an LLM-assisted workflow. A distinction to make is that most of the benchmarks ported were in C, not C++. We produced more C ports than C++ ports because the C ports were faster to implement, allowing us to cover more benchmarks. Because the C++ ports took longer to implement, there are fewer of them than the C ports. The role of the C++ ports was not to simply change the language of the benchmark, but to restore a more object-oriented structure, reduce ownership overhead, and adopt a library-like organization, even when a compact C port might otherwise simplify important parts of the original Java code.

For each of our chosen benchmarks, we first wrote a design specification describing the DaCapo workload, the upstream Java path (if applicable), the intended C or C++ architecture, the allocation behavior we wanted to preserve, and the validation checks to be performed before any measurements. These plans were then converted into implementation plans for a coding agent. Much of this work was done via Claude Code, using the Opus 4.6 model in a high-thinking configuration. The AI assistant was the main worker on porting the Java source code, while the researcher acted more as a system architect on design decisions.

As mentioned, the workflow had four stages. First, a source-discovery stage identified the actual upstream library and code that executed the DaCapo benchmark. Second, a design stage was used to guide the native architecture before an implementation plan was developed. Third, there was an implementation stage in which the port could break the large design into smaller parts for testing and validation. For the C implementations, this was more module-to-module, whereas for C++, it was more class-by-class. Fourth, a validation stage required the port to first prove that it reproduced the Java benchmark’s behavior. That validation stage compared the port against Java-generated reference outputs, not just whether it compiled or ran. We gave the model more structure so we could guide the LLM toward a more faithful port rather than merely asking it to produce a C port of a Java workload.

3.2 Validation

Validation was layered and benchmark-specific. At the lower levels, ports were checked with unit tests for parsers, data structures, numerical kernels, and other helper routines. At the workload level, each benchmark was validated against Java reference behavior using one or more of the following: golden files, byte-identical outputs, digest comparisons, structured equivalence checks, deterministic replay, and per-input oracle tables. Where possible, we also collected allocation-pressure reports to determine whether an output-valid port was still doing the same amount of work.

The main goal was to ensure that ported benchmarks were not merely plausibly equivalent but properly faithful ports, so we could run fair comparisons. However not all ports are perfect representations of their equivalent Java ports. So we do not aim to treat all the ports as equally strong evidence. Instead, we aim to interpret a result’s weakness as part of a broader trend across the benchmarks, then examine each empirical result with greater confidence.

3.3 Experimental Machine and Configurations

The GC side of the study uses DaCapo 23.11-Chopin/MR2 as the benchmark basis. The DaCapo harness supports multiple executions, with the first $n - 1$ iterations serving as a warmup and the final iteration serving as the timed iteration [4]. DaCapo supports repeated same-process iterations, but the Java reruns in our study were executed as fresh processes with single-iteration runs and a single benchmark workload thread using the flags `-n 1` and `-t 1`. We used these flags in order to avoid measuring peak steady-state behavior that a JVM can reach through repeated runs of similar allocation workloads. Prior work shows that VM benchmarks often fail to reach a consistent steady-state peak performance, and that static warm-up estimates are frequently inaccurate [25, 26]. So we see no reason to add an arbitrary fixed warmup that would have changed the targeted measurements without a guarantee of improving the results, while also making the results harder to parse, since all iterations of a run would have been done in the same process.

All measurements were collected on a remote Linux host running Ubuntu 24.04 with Linux ker-

nel 6.17.0-1012-aws on an aarch64 system. The machine exposes 16 logical CPUs, ARM Cortex-A72 cores, with 31 GiB of RAM and no swap configured. Java experiments used Ubuntu OpenJDK 21.0.10, while native binaries were compiled with GCC 13.3.0 and G++ 13.3.0. Hardware-counter collection used `perf` 6.17.13. One limitation of the setup is that the harness logs do not record the exact AWS instance type, nor do they record a language-runtime version for C or C++ beyond the compiler versions.

Java reruns in the current study were executed with DaCapo’s single-iteration, single-thread configuration (`-n 1 -t 1`), together with an explicitly selected GC and matching `-Xms/-Xmx` heap settings. No other flags were used beyond the collector and heap size choices. The primary GC sets in the harness are Serial, Parallel, G1, and ZGC.

The native side used C and C++ ports compiled with O2-level optimization. We explore different levels of optimization within RQ7. Each compiled port was linked with one of three general-purpose allocators: the default system allocator, `jemalloc`, or `mimalloc`.

The performance harness collected wall-clock time, hardware counter data, and resident set size during the same run. Resident set size was sampled every 20 ms from the Linux process, and the maximum observed `VmRSS` was recorded. Hardware-counter collection used Linux’s `perf` and collected data on cycles, instructions, page faults, cache activity, and TLB-related events. Each of these results was collected during every trial. We measured 5 trials per configuration for both Java and native code. We report the medians over successful runs, and no general outlier rule was applied.

We intentionally did not enforce CPU pinning between Java and native runs. Both were left unpinned for fairness. Some thought was given to pinning both runs to isolate concurrent factors. However, modern GCs like ZGC rely on concurrency to achieve competitive performance, whereas GCs that do not rely on concurrency do not. The fact that we keep worker threads to one for both Java and native runs means we keep concurrency out of the main work being done, thus allowing modern GCs to take advantage of their newer strengths rather than unfairly crippling them to make both explicit and managed collection non-concurrent.

3.4 Heap Selection

Heap sizes were chosen relative to a benchmark and GC-specific minimum viable heap. For each Java configuration, we performed a binary search over candidate minimum heap sizes and identified the smallest heap at which the benchmark ran stably. That stable minimum then served as the reference point for subsequent heap sweeps and runs at multiples of that size. We ran a fixed set of multiples: 1.0 $\times$, 1.25 $\times$, 1.5 $\times$, 2.0 $\times$, 3.0 $\times$, 4.0 $\times$, and 6.0 $\times$. The use of multiples of the min heap size was done to follow what was done in Hertz and Berger’s paper and how they performed their space–time tradeoff analysis [1]. The purpose is to determine whether giving Java more memory removes not only small-heap penalties, but also whether increasing heap size is sufficient to close the gap to the native baseline.

3.5 Scope and Limitations

Our methodology is narrower than the original oracle experiment. It does not preserve the exact JVM allocation sites, object identity, runtime services, or compiler pipeline of a managed execution. The results, therefore, should not be interpreted as a purely isolated measure of GC cost. Instead, they map out a modern empirical comparison between managed Java executions and workload-equivalent native baselines.

The use of Claude Code and Claude Opus 4.6 introduces the possibility of implementation bias or valid but incorrectly generated code. We tried to address potential LLM bias or simplification by making validation, manual review, and benchmark-specific equivalence testing the criteria for acceptance, rather than relying solely on model output.

4 References

- [1] Matthew Hertz and Emery D. Berger. 2005. Quantifying the performance of garbage collection vs. explicit memory management. In *Proc. OOPSLA '05*, 313–326. <https://doi.org/10.1145/1094811.1094836>
- [2] Xianglong Huang, Stephen M. Blackburn, Kathryn S. McKinley, J. Eliot B. Moss, Zhenlin Wang, and Perry Cheng. 2004. The garbage collection advantage: improving program locality. In *Proc. OOPSLA '04*, 69–80. <https://doi.org/10.1145/1028976.1028983>
- [3] Zixian Cai, Stephen M. Blackburn, Michael D. Bond, and Martin Maas. 2022. Distilling the real cost of production garbage collectors. In *Proc. ISPASS '22*, 46–57. <https://doi.org/10.1109/ISPASS55109.2022.00005>
- [4] Stephen M. Blackburn et al. 2006. The DaCapo benchmarks: Java benchmarking development and analysis. In *Proc. OOPSLA '06*, 169–190. <https://doi.org/10.1145/1167473.1167488>
- [5] Andrew Borg, Andy J. Wellings, Christopher D. Gill, and Ron K. Cytron. 2006. Real-time memory management: life and times. *Proc. ECRTS 2006*, 237–250. <https://doi.org/10.1109/ECRTS.2006.21>
- [6] Maria Carpen-Amarie, Georgios Vavouliotis, Konstantinos Tovletoglou, Boris Grot, and Rene Mueller. 2023. Concurrent GCs and modern Java workloads: a cache perspective. In *Proc. ISMM '23*. <https://doi.org/10.1145/3591195.3595269>
- [7] Matthew Hertz, Stephen M. Blackburn, J. Eliot B. Moss, Kathryn S. McKinley, and Darko Stefanović. 2006. Generating object lifetime traces with Merlin. *ACM TOPLAS* 28, 3 (2006), 476–516. <https://doi.org/10.1145/1133651.1133654>
- [8] Wenyu Zhao and Stephen M. Blackburn. 2020. Deconstructing the Garbage-First collector. In *Proc. VEE '20*, 15–29. <https://doi.org/10.1145/3381052.3381320>

- [9] Kunal Sareen and Stephen M. Blackburn. 2022. Better understanding the costs and benefits of automatic memory management. In *Proc. MPLR '22*, 29–44. <https://doi.org/10.1145/3546918.3546926>
- [10] Oracle. 2018. JEP 318: Epsilon: a no-op garbage collector (experimental). <https://openjdk.org/jeps/318>
- [11] Jason Evans. 2006. A scalable concurrent malloc(3) implementation for FreeBSD. In *Proc. BSDCan*.
- [12] Sanjay Ghemawat and Paul Menage. 2009. TCMalloc: thread-caching malloc. Google Research.
- [13] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. 2019. Mimalloc: free list sharding in action. In *Proc. APLAS '19*. https://doi.org/10.1007/978-3-030-34175-6_1
- [14] Microsoft. 2021. mi-malloc: performance benchmarks. <https://microsoft.github.io/mimalloc/bench.html>
- [15] Hyperledger Besu. 2023. malloc vs. tcmalloc vs. jemalloc vs. mimalloc memory utilization study.
- [16] Theo Ezell. 2025. G1, ZGC, and Shenandoah: OpenJDK's garbage collectors for very large heaps. IBM TechXchange Community.
- [17] Codrut Stancu, Christian Wimmer, Stefan Brunthaler, Per Larsen, and Michael Franz. 2015. Safe and efficient hybrid memory management for Java. In *Proc. ISMM '15*, 81–92. <https://doi.org/10.1145/2754169.2754185>
- [18] Aleksey Shipilev. 2020. JEP 379: Shenandoah: a low-pause-time garbage collector (production). OpenJDK. <https://openjdk.org/jeps/379>
- [19] Stefan Karlsson. 2023. JEP 439: Generational ZGC. OpenJDK. <https://openjdk.org/jeps/439>

- [20] Nathan P. Ricci, Samuel Z. Guyer, and J. Eliot B. Moss. 2013. Elephant tracks: portable production of complete and precise GC traces. In *Proc. ISMM '13*, 109–118. <https://doi.org/10.1145/2464157.2466484>
- [21] Stephen M. Blackburn, Zixian Cai, Rui Chen, Xi Yang, John Zhang, and John Zigman. 2025. Rethinking Java Performance Analysis. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*. <https://doi.org/10.1145/3669940.3707217>
- [22] Anthropic. 2026. Claude Code overview. <https://code.claude.com/docs/en/overview>
- [23] Anthropic. 2026. Introducing Claude Opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>
- [24] Anthropic. 2026. Building with extended thinking. <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
- [25] Luca Traini, Vittorio Cortellessa, Daniele Di Pompeo, and Michele Tucci. 2023. Towards effective assessment of steady state performance in Java software: Are we there yet? *Empirical Software Engineering* 28, 1. <https://doi.org/10.1007/s10664-022-10247-x>
- [26] Edd Barrett, Carl Friedrich Bolz-Tereick, Rebecca Killick, Sarah Mount, and Laurence Tratt. 2017. Virtual Machine Warmup Blows Hot and Cold. *Proceedings of the ACM on Programming Languages* 1, OOPSLA, Article 52. <https://doi.org/10.1145/3133876>